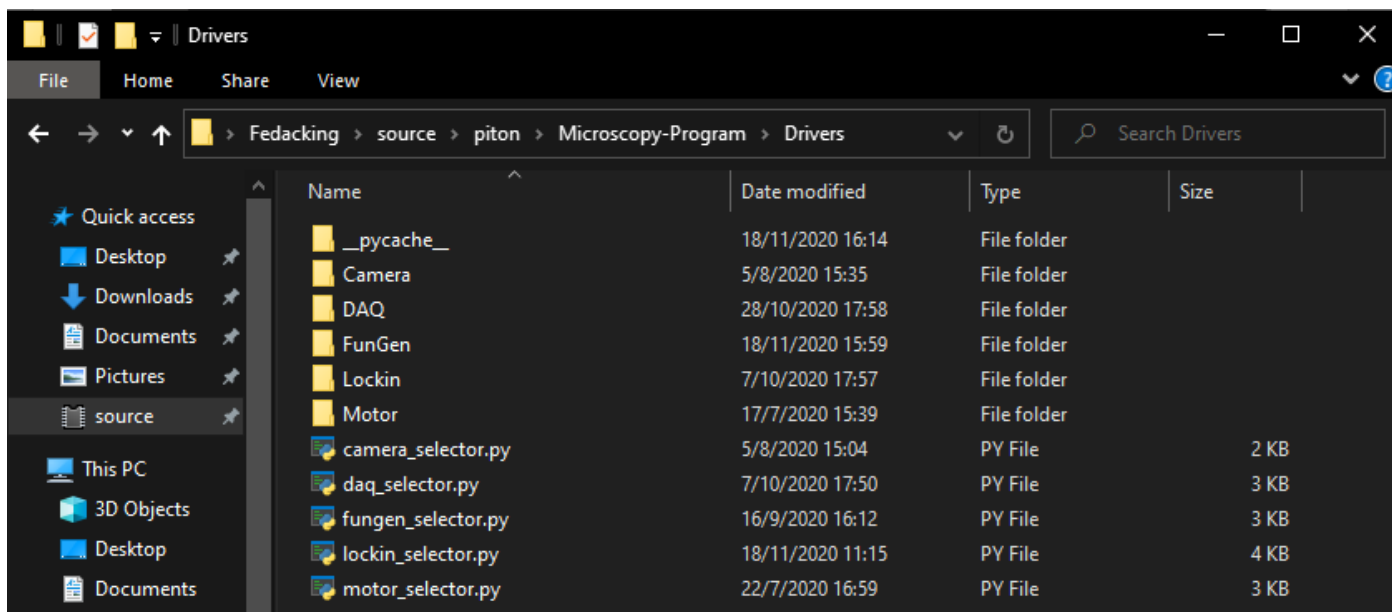
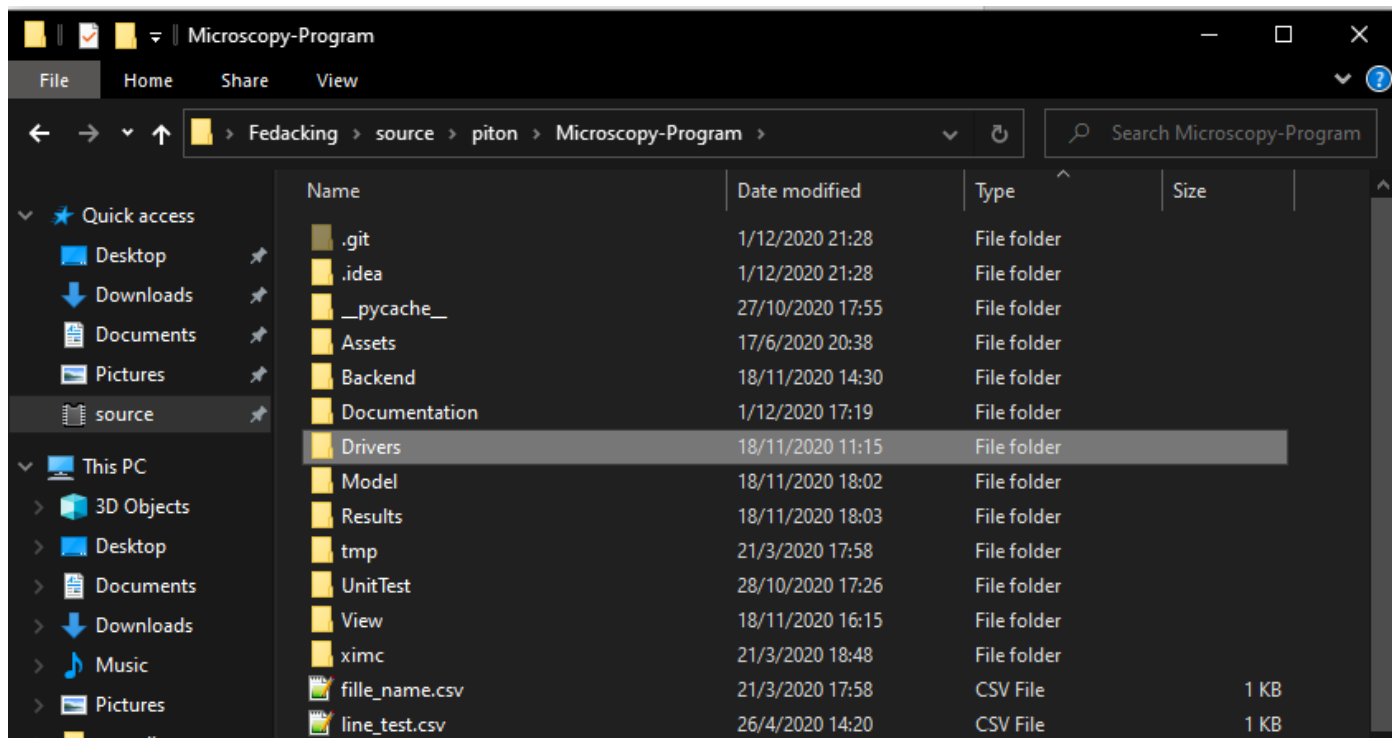


Guía de Cambio de Drivers

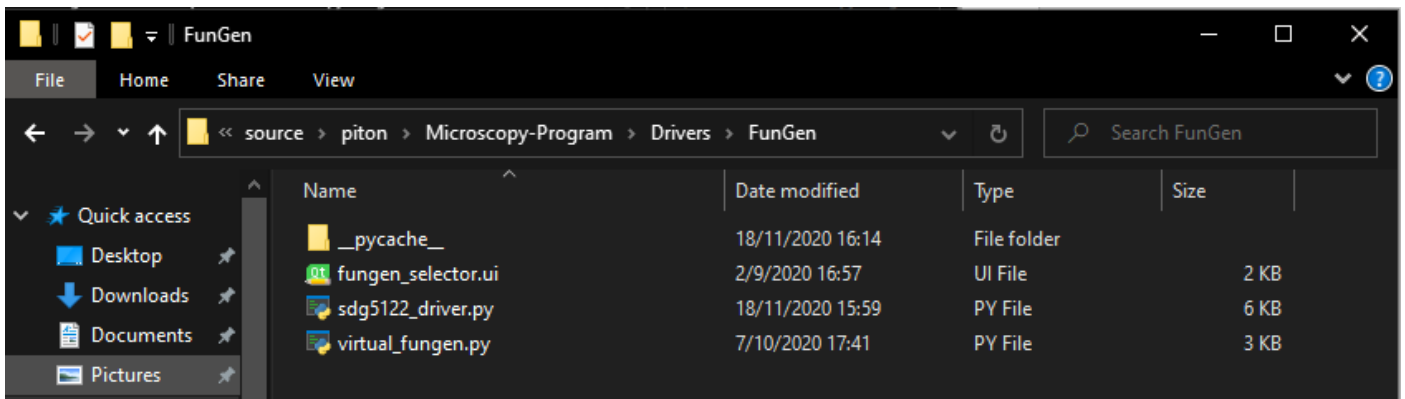
1. Estructura de los Archivos:

Los archivos de administración de dispositivos conectados se encuentran en la carpeta “Drivers” en el directorio principal.



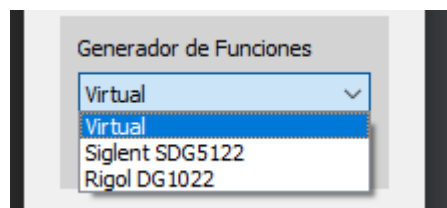
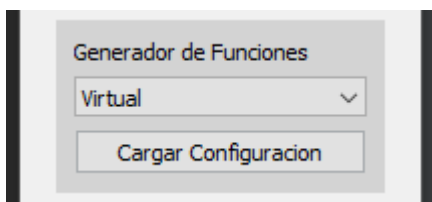
Cada tipo de dispositivo usado en el programa tiene su propia carpeta. Estas carpetas contienen en su interior todo lo necesario para hacer funcionar el dispositivo y poder elegirlo. Puntualmente es necesario tener el archivo de Python que incluye el driver de Lantz, el selector de fungen y cualquier archivo extra necesario. Por ejemplo, el driver del lockin tiene un “.dll” provisto por el Anfatec.

Ej. de Generador de Funciones:



2. Selección de Módulos

Cada dispositivo tiene una lista desplegable donde se muestran las opciones disponibles para seleccionar un dispositivo. Ej. Del generador de funciones y sus opciones:



La lista de elementos esta definido en el archivo de fungen_selector.py, en la siguiente seccion de codigo

```
def dg1022(self):
    # driver = DG1022("dummy") Cambiar al posta cuando vamos a pc
    driver = DG1022("dummy")
    driver.initialize()
    return driver

def sgd5122(self):
    driver = SDG5122("dummy")
    driver.initialize()
    return driver

def virtual(self):
    driver = VirtualFungen()
    driver.initialize()
    return driver

fungen_available = {
    "Virtual": virtual,
    "Siglent SDG5122": sgd5122,
    "Rigol DG1022": dg1022,
}
```

El diccionario de “fungen_available” contiene todos los Generadores de Funciones disponibles al usuario. Para agregar uno es necesario agregar una entrada a este diccionario. El diccionario apunta a una función, definida arriba, la cual inicializa y devuelve el driver. Esta función tiene que ser sin parámetros.

En el ejemplo mostrado arriba si la lista está en Virtual al cargar el archivo ejecuta la función “virtual” que devuelve una instancia del driver virtual.

3. Carga de Configuración

En caso de tener datos que se quieran cargar de un archivo de configuración “.cfg” el programa tiene un botón para cargar configuración. Este botón ejecuta la función “load_conf(self)” que define un parámetro interno “self.conf”.

```
def load_conf(self):
    if file_name:
        self.widget.load_conf_bt.setText(Path(file_name).name)
        self.conf = ConfigParser()
        self.conf.read(file_name)
```

Para acceder a los datos en este archivo de configuración se hace con la misma sintaxis de diccionarios en Python. Ej.: el driver del DAQ USB_2527 necesita un numero de tableta para poder conectarse, entonces se lo pasamos del archivo de configuración.

```
def usb_2527(self):
    driver = USB2527DaqBackend(self.conf["CONNECTION"]["board_number"])
    return driver
```

En caso de ser necesario este parámetro el programa avisa la falta de archivo de configuración.

Ej. del conf que viene con el daq, llamado "usb2527.cfg"

```
[CONNECTION]
board_number = 0
```

Apéndice:

La librería usada para la lectura del archivo config es "configparrser" una librería que viene estándar con las instalaciones de Python como la de Anaconda. Acá está el link a la página oficial de documentación:

<https://docs.python.org/3.7/library/configparser.html>